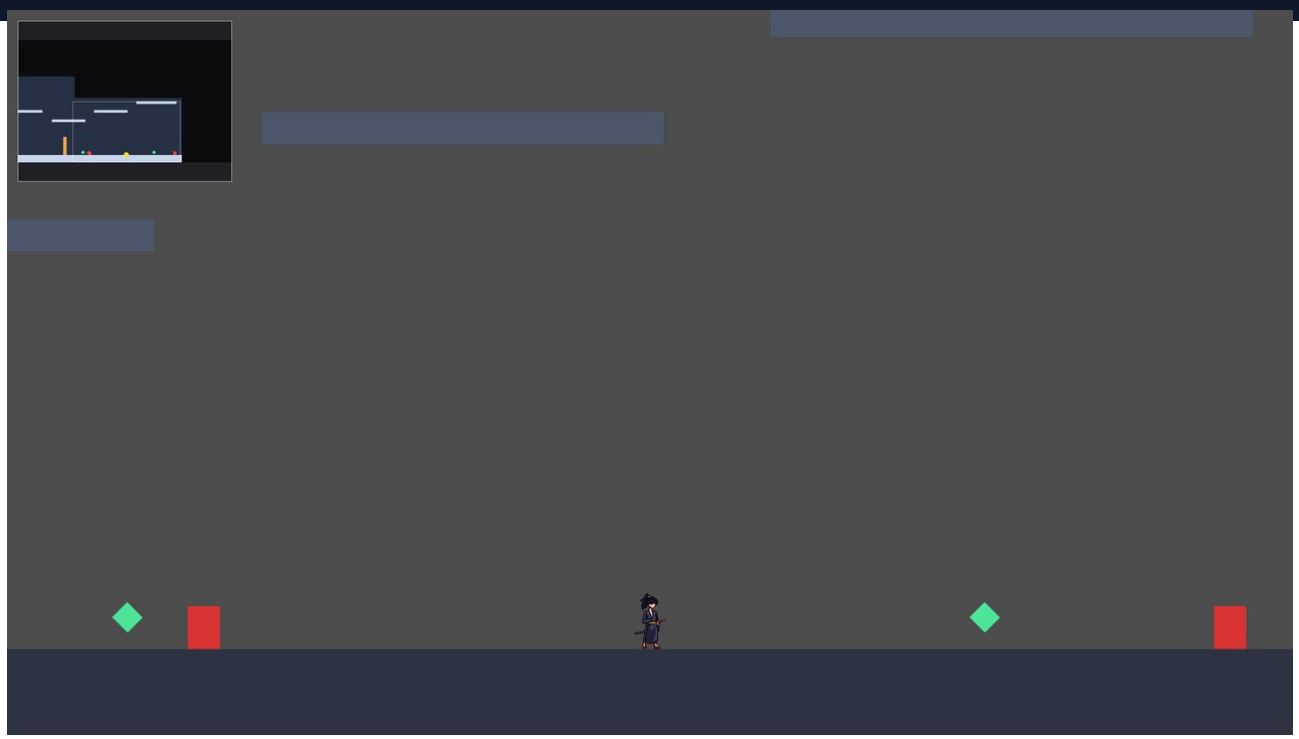
BLOCO 4 · SISTEMAS DINÂMICOS E PROCEDURAIS · LAB DE SOLUÇÕES TÉCNICAS

MapScanner v1.0

Um minimapa de exploração para jogos de plataforma 2D na Godot 4 — explicado passo a passo



Fase de teste do projeto TDE: no canto superior esquerdo, o minimapa revela a sala secreta aos poucos, depois que a parede foi destruída. Captura real do jogo (Godot 4.6.1).

Projeto	TDE (Godot 4.6.1 · 2D · GDScript)
Solução técnica	Minimapa de exploração com névoa de guerra, objetos dinâmicos, entidades temporárias, salas secretas e progresso salvo
Cenas	`map_scanner_test.tscn` (fase de teste) e `game.tscn` (fase principal)
Base	Especificação “MapScanner v1.0 — Especificação técnica e decisões arquiteturais”
Equipe	(preencher: nome dos integrantes)

Sumário

1. A solução: o que é e para que serve 3
   - 1.1 Que problemas ele resolve . . . 3
   - 1.2 Onde essa solução é útil . . . 3
2. A ideia central aplicada ao MapScanner 3
3. O jogo: a fase de teste 4
   - 3.1 Controles . . . 5
   - 3.2 Mecânicas do jogador . . . 5
   - 3.3 Objetos da fase de teste . . . 7
4. Como o minimapa funciona (mecanismo principal) 7
   - 4.1 Enquadramento: o que aparece no quadradinho . . . 7
   - 4.2 Exploração: a grade de células . . . 8
   - 4.3 Terreno estático e terreno dinâmico . . . 9
   - 4.4 Inimigos, colecionáveis e plataformas móveis . . . 10
   - 4.5 Regiões secretas . . . 12
   - 4.6 Persistência (save) . . . 14
   - 4.7 Política de atualização (desempenho) . . . 15
5. Arquitetura e scripts 16
   - 5.1 A ordem de entrada na fase . . . 16
   - 5.2 Configuração por fase: MapDefinition . . . 17
   - 5.3 O jogador (player.gd) . . . 18
6. Parâmetros que alteram o comportamento 19
7. Integração e uso: a fase principal 19
   - 7.1 Como colocar o MapScanner em qualquer fase (passo a passo) . . . 20
   - 7.2 Correções feitas no jogo durante a integração . . . 21
8. Validação e testes 21
9. Ferramentas, referências e créditos 22
10. Dificuldades, decisões e aprendizados 22
    - 10.1 Dificuldades e como foram resolvidas . . . 22
    - 10.2 Decisões e interpretações . . . 22
    - 10.3 Limitações conhecidas . . . 23
    - 10.4 Dicas para quem for implementar algo parecido . . . 23
11. Roteiro sugerido para o vídeo-showcase (~5 min) 23
12. Validação simples do tema (para o acompanhamento) 24
13. Contribuição de cada integrante 24

Como ler este documento

Este documento segue a ordem pedida na atividade: **qual é a solução** e que problema resolve; **como funciona** o mecanismo; a **implementação integrada**; os **parâmetros** que mudam o comportamento; **ferramentas e referências**; **dificuldades e aprendizados**; e a **contribuição** de cada integrante. No fim há um roteiro pronto para o vídeo-showcase de ~5 minutos.

Caixas amarelas **“Por quê?”** justificam as decisões. Blocos escuros são trechos **reais** dos scripts do projeto.

1. A solução: o que é e para que serve

O **MapScanner** é um **minimapa de exploração** para jogos de plataforma 2D. Ele fica fixo no canto superior esquerdo da tela e mostra, de forma simplificada, o terreno que o jogador **já viu**, onde ele está agora e alguns marcadores (inimigos, colecionáveis, plataformas móveis, portas e paredes).

A ideia mais importante: o minimapa **não é uma segunda câmera** “filmando” o jogo. Ele não copia a imagem da fase. Ele **lê informações** do jogo (onde está a câmera, quais objetos existem, quais colisões estão ativas) e **desenha** uma versão simplificada com retângulos. Por isso é leve e funciona em qualquer fase.

1.1 Que problemas ele resolve

- **Orientação:** em fases grandes o jogador se perde; o minimapa mostra onde ele está e o que ainda falta explorar.
- **Névoa de guerra (exploração):** só aparece o que a câmera já mostrou. Explorar vira recompensa.
- **Mundo que muda:** se uma porta abre ou uma parede é destruída, o mapa acompanha.
- **Informação temporária:** inimigos e colecionáveis vistos ficam marcados enquanto fazem sentido, sem “trapacear” mostrando o que não foi visto.
- **Segredos:** salas secretas continuam escondidas no mapa até o jogador cumprir a condição de desbloqueio.
- **Continuidade:** o que foi explorado é salvo e volta ao reabrir o jogo.

1.2 Onde essa solução é útil

Em **metroidvanias** e jogos de plataforma com exploração, em fases com áreas secretas, em jogos com **backtracking** (voltar a lugares antigos com novas habilidades) e em qualquer jogo 2D que precise de um mapa que “se desenhe sozinho” sem o designer ter que pintar uma imagem de mapa para cada fase.

Por quê?

A especificação pede que o sistema seja **reutilizável entre fases**. Se o mapa fosse uma imagem desenhada à mão, cada fase nova exigiria uma imagem nova e qualquer mudança no cenário deixaria o mapa errado. Lendo a geometria direto das colisões, o mapa sempre corresponde ao que existe de verdade na fase.

2. A ideia central aplicada ao MapScanner

A atividade resume toda solução técnica em uma cadeia. No MapScanner ela fica assim:



Figura 1 — Informações do jogo → processamento técnico → comportamento dinâmico → integração e uso.

Etapa da cadeia	No MapScanner	Exemplo concreto na fase de teste
Informações do jogo	Retângulo visível da Camera2D, posição do jogador, nós marcados com grupos (`map_*`), formas de colisão, sinais de mudança.	A porta tem uma `CollisionShape2D`; ao abrir, a colisão é desativada e a porta emite `map_changed`.
Processamento técnico	Converte o mundo numa grade de células de 64 px; decide o que foi descoberto; avalia condições de segredo; relê só o que mudou.	Toda célula que encosta no retângulo da câmera vira “descoberta”.
Comportamento dinâmico	O mapa revela áreas, marcadores aparecem/somem, portas abertas somem do mapa, a sala secreta continua escura até ser liberada.	Ao quebrar a parede, a sala secreta passa a poder ser descoberta (mas aos poucos).
Integração e uso	HUD num `CanvasLayer`, save em `user://`, funcionando na fase de teste e na fase principal.	Fechar e abrir o jogo mantém o que foi explorado.

3. O jogo: a fase de teste

A especificação pede para validar o sistema primeiro numa **cena experimental independente**, antes da fase principal. Essa cena é `Scenes/map_scanner_test.tscn`: um corredor de 4096 × 1088 pixels com chão, paredes nas pontas, sete plataformas fixas, uma plataforma móvel, uma porta, uma parede destrutível, quatro inimigos, quatro colecionáveis e uma sala secreta.

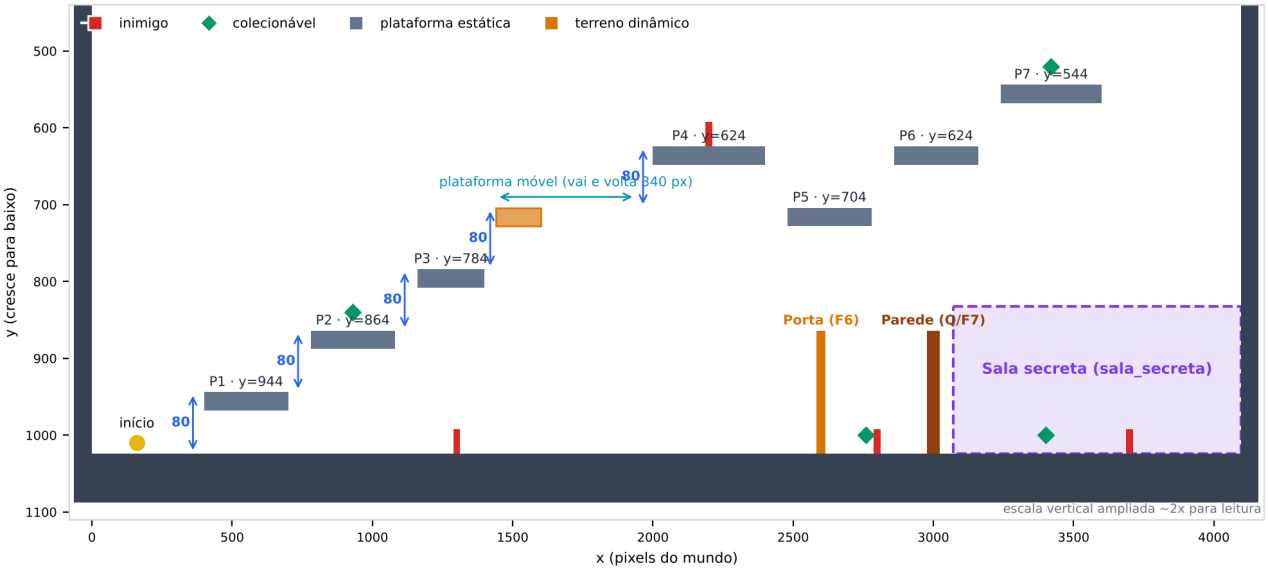


Figura 2 — Mapa da fase de teste, gerado a partir das coordenadas reais da cena. As setas azuis marcam os degraus de 80 px. A sala secreta (roxa) fica no chão, depois da parede.

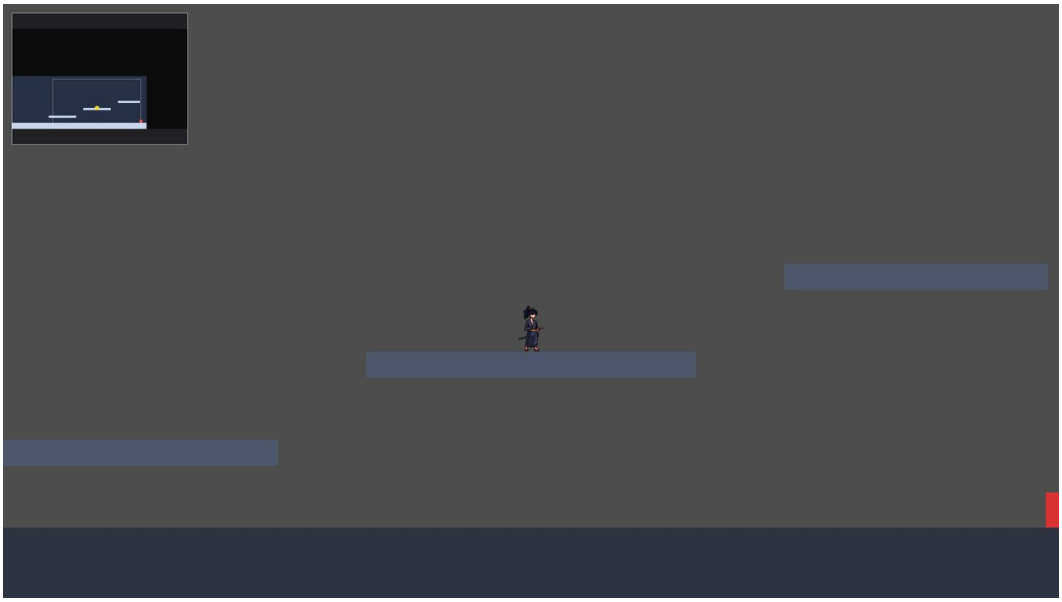


Figura 3 — O jogador na plataforma P2. O minimapa (canto) já mostra P1, P2, o chão explorado e o colecionável (losango verde).

3.1 Controles

Tecla	Ação
← / →	Andar (segure **Shift** para correr)
Espaço	Pular. No ar, pode pular **mais uma vez** (pulo duplo)
Q	Atacar: mata um inimigo próximo ou quebra a parede marrom de perto
F6	Abrir / fechar a porta (só na fase de teste)
F7	Quebrar a parede destrutível (atalho de teste)
F9	Apagar o progresso do minimapa e bloquear os segredos de novo (atalho de QA)

3.2 Mecânicas do jogador

Movimento e pulo duplo com degraus de 80 px

O jogador pula com velocidade inicial de 350 px/s para cima e gravidade de 980 px/s². Pela física, a altura máxima de um pulo é $v^2/(2g) = 350^2/(2 \cdot 980) \approx$ **62,5 px**. Pulando de novo no ponto mais alto, ele sobe mais 62,5 px: total ≈ **125 px**. No motor, medimos 65,5 px e 130,4 px (a diferença vem dos passos discretos de física, 60 por segundo).

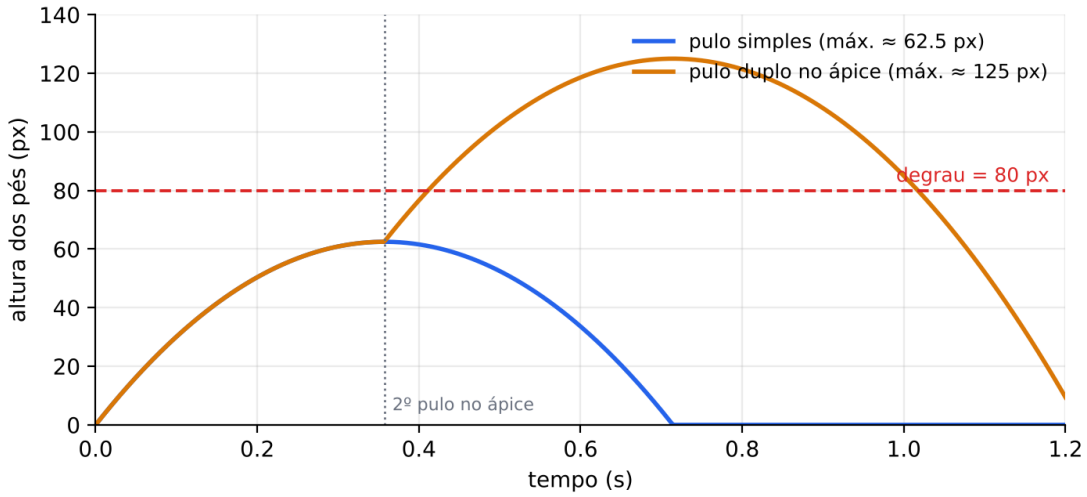


Figura 4 — Altura dos pés ao longo do tempo. A linha vermelha é o degrau de 80 px: o pulo simples não alcança; o duplo alcança com folga.

Por quê?

Os degraus foram ajustados para **80 px** (antes, a primeira plataforma ficava 136 px acima do chão, alta demais até para o pulo duplo). Com 80 px, o degrau fica **entre** o pulo simples (~60 px) e o duplo (~120 px): subir exige o pulo duplo, e isso vira uma mecânica de verdade, não um detalhe.

Cair da beirada e “coyote time”

Se o jogador sai andando de uma plataforma (sem pular), ele **perde o pulo do chão** e só tem o pulo do ar. Mas, durante 0,1 s depois de sair da beirada, ainda vale pular como se estivesse no chão (técnica conhecida como *coyote time*, comum em jogos de plataforma).

Por quê?

Antes, cair da beirada deixava os **dois** pulos disponíveis no ar — o jogador ganhava altura extra e podia pular degraus que deveriam exigir planejamento. O coyote time evita o efeito contrário: o jogador apertou o pulo um instante depois de sair da beirada e o jogo “ignorou”, o que parece injusto.

Ataque

A tecla Q toca a animação de ataque. Na fase de teste, um ataque perto de um inimigo o elimina, e perto da parede marrom a quebra.

3.3 Objetos da fase de teste

Objeto	Como funciona	O que testa no minimapa
Plataformas fixas (P1–P7)	`StaticBody2D` no grupo `map_static_terrain`	Terreno estático lido uma vez
Plataforma móvel	`AnimatableBody2D` que vai e volta 340 px (grupo `map_moving_platforms`)	Aparece só enquanto a câmera vê; só contorno; sem rastro
Porta	F6 desativa a colisão e esconde o visual; emite `map_changed`	Atualização de terreno por evento
Parede destrutível	Some (`queue_free`) com F7 ou ataque próximo	Remoção de objeto + condição de segredo
Inimigos (4)	Patrulham de um lado para o outro; morrem com ataque	Última posição conhecida, invalidação, morte
Colecionáveis (4)	`Area2D`: somem quando o jogador encosta	Marcador da sessão; some ao coletar
Sala secreta	Região roxa depois da parede (`SecretArea`)	Mascaramento e desbloqueio por condições E/OU

4. Como o minimapa funciona (mecanismo principal)

4.1 Enquadramento: o que aparece no quadradinho

- Fica no canto superior esquerdo, com proporção **4:3**. Referência: 320 × 240 px numa tela 1920 × 1080; em telas menores ele escala (com limites mínimo e máximo).
- Cobre pelo menos **2× a largura E 2× a altura** do que a câmera mostra. Como câmera e minimapa têm proporções diferentes, aumenta-se o eixo necessário mantendo a escala uniforme (sem esticar o mapa).
- A janela segue o jogador, com **suavização vertical** (para não tremer a cada pulo) e sem sair dos limites da fase. Perto das bordas, o marcador do jogador sai do centro — mas sempre mostra a **posição verdadeira**.
- Se a fase é menor que a área coberta, a escala é mantida, o mapa fica centralizado e o resto recebe fundo neutro (é o caso da fase principal).

minimap_renderer.gd — janela do mundo mostrada no minimapa

```
func _compute_window(camera_size: Vector2) -> Rect2:
    var aspect := _definition.hud_reference_size.x / _definition.hud_reference_size.y
    var w := maxf(camera_size.x * 2.0, camera_size.y * 2.0 * aspect)
    var h := w / aspect
    var bounds := _definition.world_bounds
    var cx := _center_on_axis(_player_pos.x, w, bounds.position.x, bounds.size.x)
    var cy := _center_on_axis(_smooth_y, h, bounds.position.y, bounds.size.y)
    return Rect2(Vector2(cx - w * 0.5, cy - h * 0.5), Vector2(w, h))

static func _center_on_axis(target: float, extent: float, bounds_start: float, bounds_len: float) -> flo
at:
    if bounds_len <= extent:
        return bounds_start + bounds_len * 0.5
    return clampf(target, bounds_start + extent * 0.5, bounds_start + bounds_len - extent * 0.5)
```

Por quê?

`maxf(camera.x * 2, camera.y * 2 * aspect)` escolhe o eixo que precisa crescer para cumprir “pelo menos 2× nos dois eixos” sem distorcer. `_center_on_axis` resolve as duas regras de borda de uma vez: se a fase é maior que a janela, faz *clamp* (a janela encosta na borda); se é menor, centraliza.

4.2 Exploração: a grade de células

O mundo é dividido numa **grade lógica** de células (64 × 64 px nas duas fases). A cada 0,1 s o sistema pega o retângulo visível da câmera e marca como **descoberta** toda célula que encosta nele. Descoberta é permanente: a célula nunca é “esquecida”. Não há teste de oclusão — se a câmera mostrou, conta como visto, mesmo atrás de uma parede.

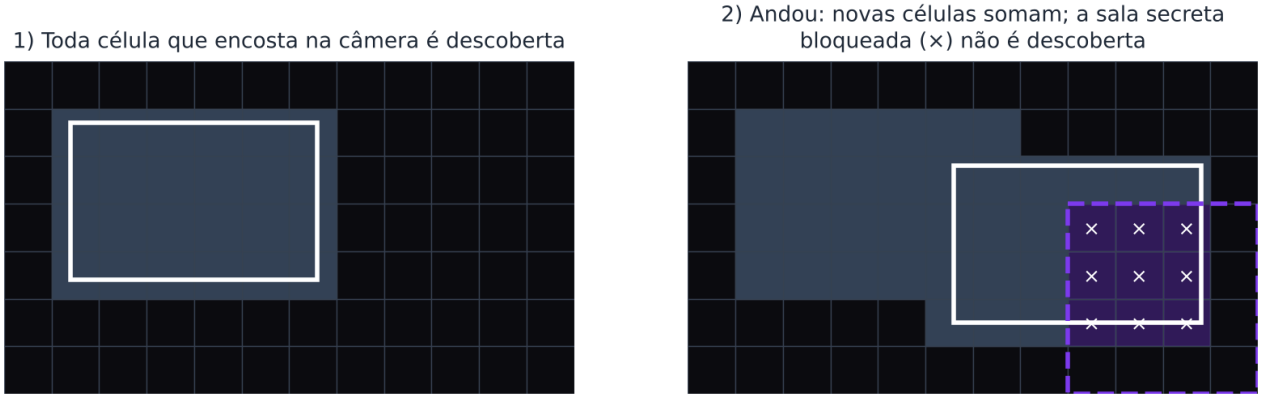


Figura 5 — A câmera (branco) descobre as células que toca. Células de uma sala secreta bloqueada (×) não são descobertas, mesmo dentro da câmera.

exploration_tracker.gd — observe()

```
func observe(view_rect: Rect2) -> int:
    var visible := view_rect.intersection(_definition.world_bounds)
    if visible.size.x <= 0.0 or visible.size.y <= 0.0:
        return 0

    var cs := float(_definition.cell_size)
    var first := Vector2i(floori(visible.position.x / cs), floori(visible.position.y / cs))
    var last := Vector2i(ceili(visible.end.x / cs) - 1, ceili(visible.end.y / cs) - 1)
    var cell_range := Rect2i(first, last - first + Vector2i.ONE)

    # Enquadramento não mudou de célula: nada a recalcular.
    if _has_last_range and cell_range == _last_range:
        return 0
    _last_range = cell_range
    _has_last_range = true

    var found: Array[Vector2i] = []
    for y in range(first.y, last.y + 1):
        for x in range(first.x, last.x + 1):
            var cell := Vector2i(x, y)
            if _discovered.has(cell):
                continue
            if authorizer.is_valid() and not authorizer.call(cell):
                continue
            _discovered[cell] = true
            found.append(cell)

    if not found.is_empty():
        cells_discovered.emit(found)
    return found.size()
```

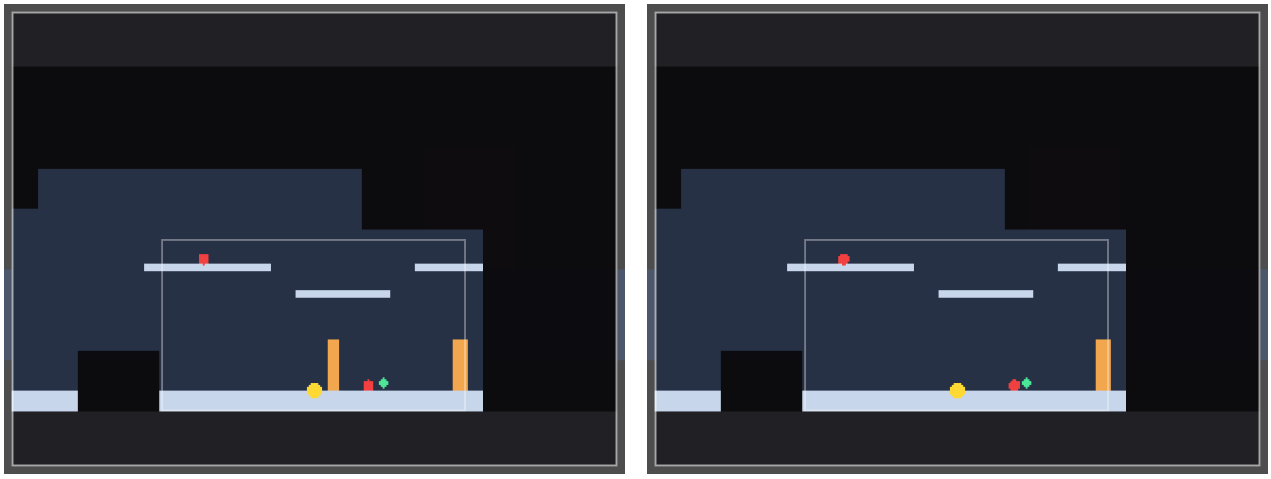
Por quê?

Grade em vez de pixels: guardar “quais células já vi” é pequeno (um dicionário de coordenadas) e fácil de salvar. **Cache do enquadramento** (`_last_range`): se a câmera não mudou de célula, nada é recalculado. **Células já conhecidas são puladas**: a especificação pede para não reprocessar o que já se sabe. O `authorizer` é o “gancho” que os segredos usam para vetar células.

4.3 Terreno estático e terreno dinâmico

Estático (chão, paredes, plataformas fixas): lido **uma vez** ao entrar na fase. Cada colisão vira um retângulo e é recortada em pedaços por célula, para o desenho só olhar as células visíveis. Funciona com `TileMapLayer` e com `StaticBody2D`.

Dinâmico (portas, paredes destrutíveis, mecanismos): atualizado **por eventos**. O objeto avisa pelo sinal `map_changed` e só ele é relido; se for destruído, sai do registro. Mudanças em terreno já conhecido aparecem mesmo fora da câmera, mas **nunca revelam** lugares ainda não descobertos.



Porta fechada: barra laranja à esquerda

Porta aberta (F6): a barra some do mapa

Figura 6 — Minimapa real. Em laranja, terreno dinâmico (porta e parede).

terrain_scanner.gd — registro e releitura por evento

```
func register_dynamic(node: Node) -> bool:
    if not node.is_in_group(GROUP_DYNAMIC_TERRAIN):
        return false
    var id := node.get_instance_id()
    if _dynamic.has(id):
        return false
    var cb := _on_dynamic_changed.bind(id)
    _dynamic[id] = {"ref": weakref(node), "rects": _dynamic_rects_of(node), "cb": cb}
    if node.has_signal(CHANGE_SIGNAL):
        node.connect(CHANGE_SIGNAL, cb)
    _rebuild_cache()
    return true

func flush_dynamic() -> int:
    if _dirty.is_empty():
        return 0
    var n := 0
    for id: int in _dirty:
        if not _dynamic.has(id):
            continue
        var node := (_dynamic[id]["ref"] as WeakRef).get_ref() as Node
        if node == null:
            _dynamic.erase(id)
            continue
        _dynamic[id]["rects"] = _dynamic_rects_of(node)
        n += 1
    _dirty.clear()
    _rebuild_cache()
    return n
```

Test/test_door.gd — a porta avisa o minimapa

```
func _aplicar() -> void:
    if not is_node_ready():
        return
    for c in find_children("*", "CollisionShape2D", true, false):
        (c as CollisionShape2D).set_deferred("disabled", aberta)
    for v in find_children("*", "CanvasItem", false, false):
        if not v is CollisionShape2D:
            (v as CanvasItem).visible = not aberta
    # Emitido depois do set_deferred, para o minimapa ler a colisão já atualizada.
    map_changed.emit.call_deferred()
```

Por quê?

Reler todos os objetos a cada frame funcionaria, mas a especificação pede **atualização orientada a eventos**, sem reconstrução completa a cada frame. Nos testes, 60 frames sem eventos = **zero** reconstruções.

Detalhe importante: a porta desativa a colisão com `set_deferred` (exigência da Godot durante a física). Se o sinal fosse emitido na hora, o minimapa leria a colisão *antiga*. Por isso o sinal também é adiado (`emit.call_deferred()`): ele chega depois da colisão ser atualizada.

4.4 Inimigos, colecionáveis e plataformas móveis

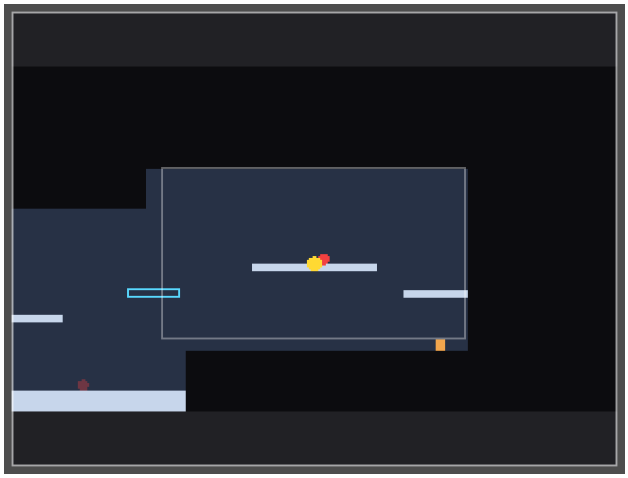
Tudo aqui é **memória temporária**: nunca vai para o save e é esquecido ao reiniciar a fase.

Tipo	Aparece quando…	Fora da câmera…	Some quando…
Inimigo (círculo vermelho)	a câmera o vê	fica na **última posição conhecida**, translúcido	morre, ou a câmera volta à última posição e ele não está lá
Colecionável (losango verde)	a câmera o vê	continua marcado durante a sessão	é coletado (pode ser redescoberto se reaparecer)

Tipo	Aparece quando…	Fora da câmera…	Some quando…
Plataforma móvel (contorno ciano)	a câmera a vê	é **ocultada** (não deixa trajetória)	sai da câmera



Inimigo visível: círculo vermelho forte



Jogador longe: o mesmo inimigo fica translúcido na última posição

Figura 7 — Minimapa real. Repare no contorno ciano da plataforma móvel.

entity_tracker.gd — detect() (roda a cada 0,1 s)

```
func detect(view_rect: Rect2) -> void:
    var view := view_rect.intersection(_definition.world_bounds)
    for id: int in _registered.keys():
        var reg: Dictionary = _registered[id]
        var node := (reg["ref"] as WeakRef).get_ref() as Node2D
        if node == null:
            _registered.erase(id)
            _observed.erase(id)
            continue
        var kind: int = reg["kind"]
        var seen := _is_observed(node, kind, view)

        if seen:
            _observed[id] = {"kind": kind, "pos": node.global_position, "visible": true,
                "rects": _platform_rects(node) if kind == Kind.PLATFORM else []}
            continue
        if not _observed.has(id):
            continue

        var obs: Dictionary = _observed[id]
        match kind:
            Kind.PLATFORM:
                _observed.erase(id) # ocultada ao sair da câmera, sem trajetória
            Kind.COLLECTIBLE:
                if not node.is_visible_in_tree():
                    _observed.erase(id) # coletado (escondido)
                else:
                    obs["visible"] = false
            Kind.ENEMY:
                if not node.is_visible_in_tree():
                    _observed.erase(id)
                elif view.has_point(obs["pos"]):
                    _observed.erase(id) # revisitou a última posição e ele não estava lá
                else:
                    obs["visible"] = false
```

Por quê?

Por que não apagar o inimigo depois de alguns segundos? Porque o jogador "lembra" onde viu o inimigo; apagar por tempo seria arbitrário. A especificação diz que o registro só é invalidado quando a câmera **volta àquele lugar** e o inimigo não está lá — exatamente como uma pessoa atualizaria a própria memória.

Por que detectar a cada 0,1 s e não a cada frame? Descobrir quem está visível é a parte cara; atualizar a posição dos que já estão visíveis é barato. Por isso `detect()` é periódico e `update_visuals()` roda todo frame (o marcador se move liso).

4.5 Regiões secretas

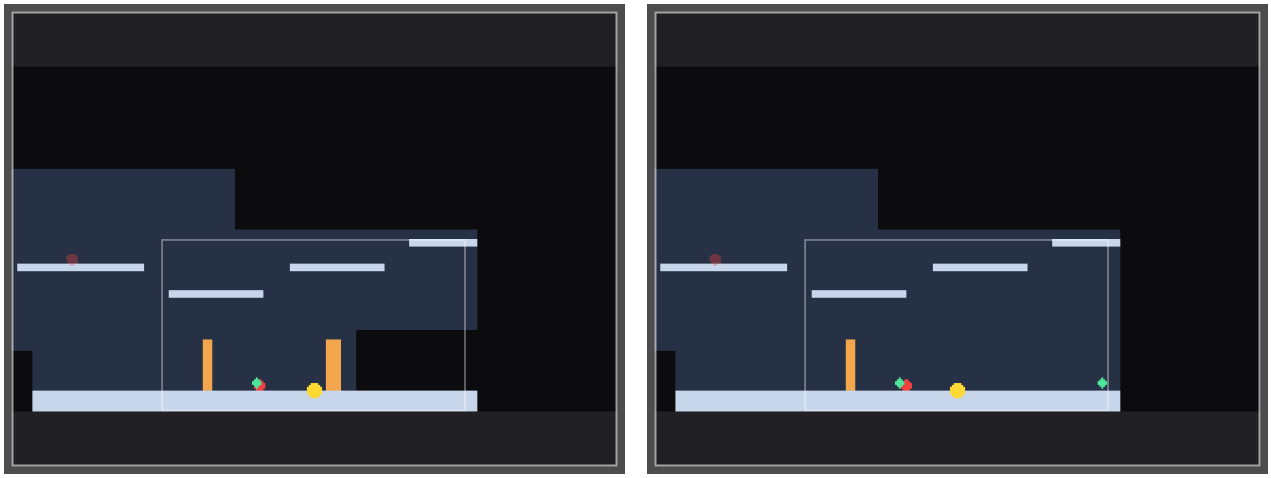
Cada segredo tem um **ID estável**, uma região e uma **condição** de desbloqueio. As condições podem ser combinadas como numa expressão lógica:

- **ENTRADA_FISICA** — o jogador entrou numa área.
- **NO_REMOVIDO** — um objeto não existe mais (ex.: a parede foi destruída).
- **PROPRIEDADE** — uma propriedade de um objeto tem um valor (ex.: `Porta.aberta == true`).
- **TODAS** (E / AND) e **QUALQUER** (OU / OR) — combinam outras condições.

Exemplo real

Condição da `sala_secreta` na fase de teste:
`QUALQUER( NO_REMOVIDO(ParedeDestrutivel), TODAS( PROPRIEDADE(Porta.aberta == true), ENTRADA_FISICA(área depois da porta) ) )`
Em português: **a sala libera se a parede cair OU se a porta estiver aberta E o jogador tiver passado por ela.**

Enquanto bloqueada, as células da sala **não são descobertas** (nem com a câmera em cima) e inimigos/colecionáveis lá dentro não aparecem. Ao desbloquear, nada é revelado de uma vez: a câmera descobre a sala normalmente, aos poucos.



Bloqueada: o chão do mapa termina na parede, embora a câmera (retângulo claro) já cubra a sala

Mesmo lugar, logo após quebrar a parede: o chão da sala e o colecionável passam a ser descobertos

Figura 8 — Minimapa real antes e depois do desbloqueio, com o jogador parado no mesmo ponto.

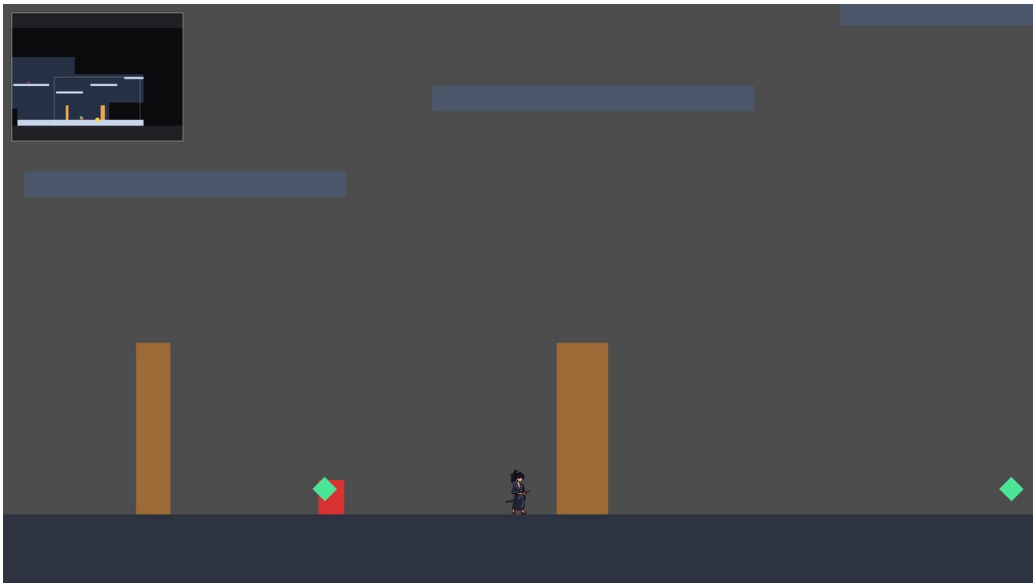


Figura 9 — A tela inteira da Figura 8 (esquerda): o colecionável à direita está dentro da sala secreta e aparece na tela, mas não no minimapa.

secret_condition.gd — avaliação recursiva das condições

```
func evaluate(level_root: Node, player_pos: Vector2, entered: Dictionary) -> bool:
    match type:
        Type.ENTRADA_FISICA:
            var key := get_instance_id()
            if not entered.has(key) and area.has_point(player_pos):
                entered[key] = true
            return entered.has(key)
        Type.NO_REMOVIDO:
            return level_root != null and not level_root.has_node(node_path)
        Type.PROPRIEDADE:
            if level_root == null or not level_root.has_node(node_path):
                return false
            return level_root.get_node(node_path).get(property) == expected
        Type.TODAS:
            var all_ok := not children.is_empty()
            for c: Resource in children:
                # avalia todas para que as entradas físicas sejam registradas
                all_ok = c.call(&"evaluate", level_root, player_pos, entered) and all_ok
            return all_ok
        Type.QUALQUER:
            var any_ok := false
            for c: Resource in children:
                any_ok = c.call(&"evaluate", level_root, player_pos, entered) or any_ok
            return any_ok
    return false
```

secret_region_manager.gd — evaluate()

```
func evaluate(player_pos: Vector2) -> Array[StringName]:
    var unlocked: Array[StringName] = []
    if _locked.is_empty():
        return unlocked
    if not _dirty and player_pos.is_equal_approx(_last_player_pos):
        return unlocked
    _dirty = false
    _last_player_pos = player_pos
    for id: StringName in _locked.keys():
        var s: SecretDefinition = _locked[id]
        if s.condition != null and s.condition.evaluate(_level_root, player_pos, _entered):
            _locked.erase(id)
            unlocked.append(id)
    if not unlocked.is_empty():
        _rebuild_blocked_cells()
        for id in unlocked:
            secret_unlocked.emit(id)
    return unlocked
```

Por quê?

Condições como recursos (Resource) aninhados: o designer monta E/OU no Inspector, sem programar. **Consulta ao gameplay em vez de cópia:** "a porta está aberta?" é perguntado à própria porta toda vez; o MapScanner não guarda o estado da porta, então nunca fica desatualizado (a especificação proíbe duplicar esses estados). **Só reavalia se algo aconteceu** (evento ou jogador se moveu).

Por que TODAS avalia todos os filhos (sem parar no primeiro falso)? Para que a entrada física seja **registrada** mesmo que a porta ainda esteja fechada — senão o jogador teria que passar de novo depois de abrir a porta.

4.6 Persistência (save)

- Um único progresso, sem slots, no arquivo `user://map_progress.json`, organizado pelo ID estável de cada fase.
- Guarda **só** o permanente: a grade explorada e os segredos completamente descobertos. Inimigos, colecionáveis e condições intermediárias **não** entram.
- Grava a cada 30 s, ao sair da fase, ao fechar o jogo e na hora em que um segredo é desbloqueado.
- **Gravação segura:** escreve num `.tmp`, confere se ele pode ser lido de volta, move o arquivo atual para `.bak` e só então promove o `.tmp`. Se o principal estiver corrompido, usa o `.bak`.
- Arquivo **versionado** (versão 2). A versão 1 (sem segredos) continua sendo lida.

map_persistence_manager.gd — trecho de save_now()

```
    # Só promove o .tmp se ele puder ser lido de volta.
    if _read_file(TMP_PATH).is_empty():
        push_error("MapPersistenceManager: gravação temporária inválida; arquivo anterior mantido.")
        return false

    if FileAccess.file_exists(SAVE_PATH):
        if FileAccess.file_exists(BAK_PATH):
            DirAccess.remove_absolute(BAK_PATH)
        DirAccess.rename_absolute(SAVE_PATH, BAK_PATH)
    var err := DirAccess.rename_absolute(TMP_PATH, SAVE_PATH)
    if err != OK:
        push_error("MapPersistenceManager: falha ao promover o arquivo (erro %d); o .bak guarda o último
 progresso válido." % err)
        return false

    _dirty = false
```

Por quê?

Se o jogo travar ou a energia cair **no meio** de uma gravação, um arquivo escrito pela metade apagaria todo o progresso. Escrevendo primeiro numa cópia temporária, o arquivo bom só é substituído quando o novo está comprovadamente íntegro. A especificação pede “gravação segura” e recuperação pelo último arquivo válido.

4.7 Política de atualização (desempenho)

Frequência	O que acontece	Por quê
Todo frame	Marcador do jogador, deslocamento da janela, posição dos inimigos e plataformas já observados	É visual e barato; precisa ser liso
A cada 0,1 s	Descoberta de células, quais entidades estão visíveis, avaliação de segredos	Mais caro; 10 vezes por segundo é imperceptível
Por evento	Porta/parede mudou, objeto criado/removido, coleta, morte, desbloqueio	Só trabalha quando algo realmente muda

map_scanner.gd — _process(): a política na prática

```
func _process(delta: float) -> void:
    var camera := get_viewport().get_camera_2d()

    # Observação só fica ativa com jogador, Camera2D e limites válidos.
    if _active and (not is_instance_valid(_player) or camera == null):
        _active = false
    if not _active:
        _try_activate(camera)
        return

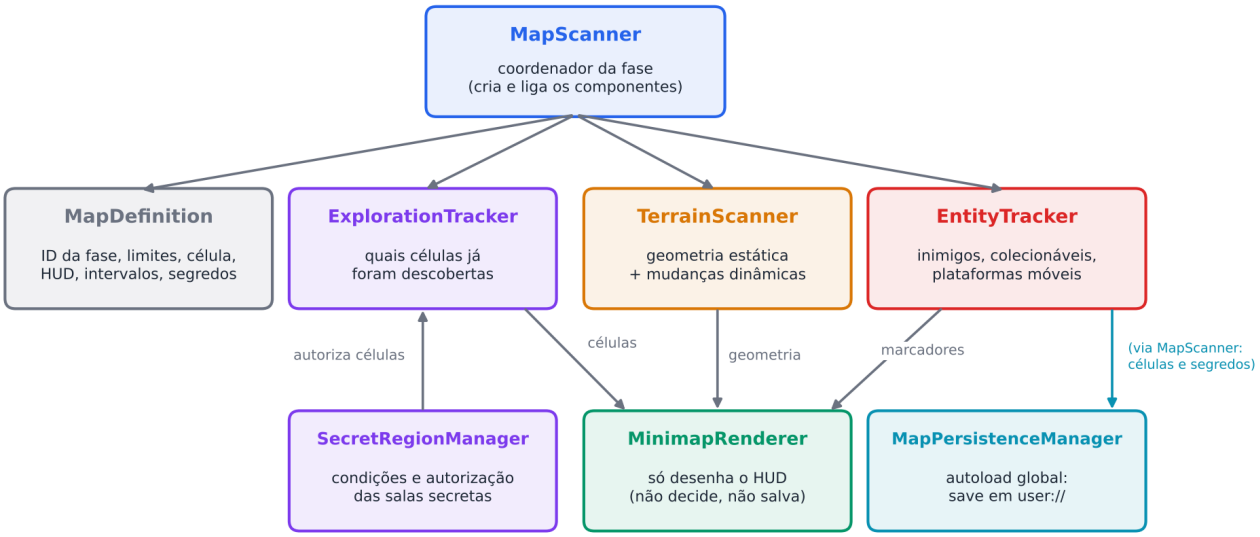
    var camera_rect := _camera_rect(camera)

    # Periódico: descoberta de células, entidades observadas, segredos.
    _detect_timer -= delta
    if _detect_timer <= 0.0:
        _detect_timer = definition.detection_interval
        secrets.evaluate(_player.global_position)
        tracker.observe(camera_rect)
        entities.detect(camera_rect)

    # Todo frame: só o que é barato e visual.
    terrain.flush_dynamic() # não faz nada se nenhum objeto avisou mudança
    entities.update_visuals()
    renderer.update_view(_player.global_position, camera_rect, delta)
```

5. Arquitetura e scripts

Cada responsabilidade é um **nó independente**, criado como filho do `MapScanner`. Assim cada parte pode ser entendida, testada e trocada sozinha. O desenho (`MinimapRenderer`) não decide nada e não salva nada; o save (`MapPersistenceManager`) não conhece nós da fase.



Setas: quem usa/consulta quem. Cada componente é um nó filho independente do MapScanner; o save é um autoload global (spec, seção 7).

Figura 10 — Componentes e quem consulta quem.

Script	Responsabilidade
`map_scanner.gd`	Coordenador local de cada fase: cria os componentes na ordem certa, registra objetos, ativa a observação quando câmera e limites são válidos, encaminha eventos e pede o save na saída.
`map_definition.gd`	Recurso de configuração por fase (ID estável, limites do mundo, tamanho da célula, HUD, intervalos, segredos). Tudo ajustável no Inspector.
`exploration_tracker.gd`	Decide e guarda as células descobertas.
`terrain_scanner.gd`	Lê o terreno estático uma vez e o dinâmico por eventos.
`entity_tracker.gd`	Registro e observação temporária de inimigos, colecionáveis e plataformas móveis.
`secret_condition.gd`, `secret_definition.gd`, `secret_area.gd`	Dados de um segredo: condição (E/OU), definição (ID + região) e área desenhada no editor (roxa).
`secret_region_manager.gd`	Avalia condições e autoriza (ou não) a descoberta de células.
`minimap_renderer.gd`	Só desenha o HUD.
`map_persistence_manager.gd`	Autoload global: lê e grava o progresso em `user://`.
`player.gd`	Movimento, pulo duplo, coyote time, ataque e retorno ao início se cair do mapa.
`Test/*.gd`	Objetos só da fase de teste: inimigo, colecionável, porta, parede destrutível, plataforma móvel.

5.1 A ordem de entrada na fase

A especificação define a ordem: ler a definição → restaurar progresso → registrar objetos → preparar geometria → só então observar. O `_ready()` segue exatamente isso:

map_scanner.gd — _ready()

```
func _ready() -> void:
    # 1) Ler a MapDefinition.
    if definition == null:
        push_warning("MapScanner: nenhuma MapDefinition atribuída; usando valores padrão.")
        definition = MapDefinition.new()
    _level_root = owner if owner != null else get_parent()

    tracker = ExplorationTracker.new()
    tracker.name = "ExplorationTracker"
    add_child(tracker)
    tracker.setup(definition)

    # 2) Restaurar progresso (células e segredos completos).
    var unlocked: Array[StringName] = []
    if use_persistence:
        _persistence = get_node_or_null("/root/MapPersistenceManager")
        if _persistence == null:
            push_warning("MapScanner: autoload 'MapPersistenceManager' não registrado; o progresso não s
erá salvo.")
        else:
            tracker.restore_cells(_persistence.get_map_cells(definition.map_id, definition.cell_size))
            unlocked = _persistence.get_unlocked_secrets(definition.map_id)
            tracker.cells_discovered.connect(_on_cells_discovered)

    secrets = SecretRegionManager.new()
    secrets.name = "SecretRegionManager"
    add_child(secrets)
    secrets.setup(definition, _level_root, unlocked)
    secrets.secret_unlocked.connect(_on_secret_unlocked)
    tracker.authorizer = secrets.is_cell_authorized

    # 3) Registrar objetos e 4) preparar geometria.
    terrain = TerrainScanner.new()
    terrain.name = "TerrainScanner"
    add_child(terrain)
    terrain.setup(definition)
    terrain.scan_static()
    terrain.dynamic_changed.connect(secrets.request_evaluation)

    entities = EntityTracker.new() # observações temporárias: morrem com a cena
    entities.name = "EntityTracker"
    add_child(entities)
    entities.setup(definition, secrets, terrain)

    for group in [TerrainScanner.GROUP_DYNAMIC_TERRAIN, EntityTracker.GROUP_ENEMIES,
            EntityTracker.GROUP_COLLECTIBLES, EntityTracker.GROUP_MOVING_PLATFORMS]:
        for node in get_tree().get_nodes_in_group(group):
            register_object(node)
    get_tree().node_added.connect(_on_node_added)
    # (...)
```

Por quê?

Restaurar antes de observar: se a observação começasse antes, o primeiro frame poderia "descobrir" células e só depois carregar o save, misturando estados. **Segredos antes do terreno:** o `authorizer` já precisa existir quando a primeira célula for avaliada. **Registro híbrido:** uma varredura inicial dos grupos pega o que já está na fase; os sinais `node_added`/`node_removed` pegam o que é criado ou destruído depois (ex.: um inimigo que nasce no meio da fase), sem registrar nada duas vezes.

5.2 Configuração por fase: MapDefinition

map_definition.gd

```
@export var map_id: StringName = &"fase_01"

## Limites do mundo configurados manualmente (em pixels, coordenadas globais).
## A janela do minimapa nunca sai daqui e células fora daqui nunca são descobertas.
@export var world_bounds: Rect2 = Rect2(0, 0, 4096, 1088)

## Tamanho da célula lógica de exploração, em pixels do mundo (pendente na spec).
@export_range(8, 512, 1) var cell_size: int = 64

@export_group("HUD")
## Tamanho de referência do minimapa (4:3) na resolução de referência.
@export var hud_reference_size: Vector2 = Vector2(320, 240)
@export var hud_reference_resolution: Vector2 = Vector2(1920, 1080)
## Margem em relação ao canto superior esquerdo, na escala de referência.
@export var hud_margin: float = 16.0
## Limites de escala do HUD (pendentes na spec).
@export_range(0.1, 4.0, 0.05) var hud_min_scale: float = 0.5
@export_range(0.1, 4.0, 0.05) var hud_max_scale: float = 1.5

@export_group("Atualização")
## Intervalo, em segundos, da checagem de descoberta de células (pendente na spec).
@export_range(0.01, 1.0, 0.01) var detection_interval: float = 0.1
## Suavização vertical da janela do minimapa. 0 = sem suavização; maior = mais rápido.
@export_range(0.0, 60.0, 0.5) var vertical_smoothing: float = 8.0

@export_group("Segredos")
## Regiões secretas da fase (ID estável + região + condições de desbloqueio).
@export var secrets: Array[SecretDefinition] = []
```

Por quê?

O **ID estável** é a chave do save: se mudar depois de lançado, o jogador perde o progresso daquela fase. Os **limites são manuais** (não calculados) porque a especificação pede limites configurados em X e Y — o designer decide até onde a janela pode ir. Tudo que a especificação marca como “pendente” virou parâmetro no Inspector.

5.3 O jogador (player.gd)

player.gd — gravidade, pulo duplo e coyote time

```
    if is_on_floor():
        # Ao tocar o chão, os pulos são restaurados.
        pulos_feitos = 0
        _tempo_fora_do_chao = 0.0
    else:
        # Aplica gravidade.
        velocity.y += gravidade * delta
        _tempo_fora_do_chao += delta
        # Saiu da beirada sem pular: passado o tempo de coyote, o pulo do chão
        # é perdido e só sobra o pulo no ar (senão cair da plataforma dava 2 pulos extras).
        if pulos_feitos == 0 and _tempo_fora_do_chao > tempo_coyote:
            pulos_feitos = 1

    # Executa o pulo (no chão ou no ar, até o limite de max_pulos).
    if Input.is_action_just_pressed("Pular") and pulos_feitos < max_pulos:
        velocity.y = forca_pulo
        pulos_feitos += 1
```

Por quê?

`pulos_feitos` conta quantos pulos já foram usados desde que saiu do chão. Pular do chão gasta 1; no ar gasta mais 1 (limite `max_pulos = 2`). Se ele **caiu** sem pular, depois do tempo de coyote o contador vira 1 — o pulo do chão foi “perdido”, sobra só o do ar.

6. Parâmetros que alteram o comportamento

Todos podem ser mudados no Inspector da Godot — ótimo para a demonstração no vídeo.

Parâmetro (onde)	Padrão	O que muda
`cell_size` (MapDefinition)	64 px	Precisão da névoa. Menor = revela de forma mais fina, mas guarda mais células.
`world_bounds` (MapDefinition)	fase de teste: 4096×1088 principal: 1152×648	Até onde a janela do minimapa anda e onde células podem ser descobertas.
`hud_reference_size`	320×240	Tamanho do minimapa numa tela 1920×1080 (sempre 4:3).
`hud_min_scale / hud_max_scale`	0,5 / 1,5	Limites de escala do HUD em telas diferentes.
`hud_margin`	16 px	Distância do canto da tela.
`detection_interval`	0,1 s	Frequência da descoberta de células, detecção de entidades e avaliação de segredos.
`vertical_smoothing`	8	Suavização vertical da janela. 0 = segue seco; maior = acompanha mais rápido.
`secrets` (MapDefinition)	1 na fase de teste	Lista de segredos com ID, região e condição E/OU.
`save_interval` (MapPersistenceManager)	30 s	Intervalo da gravação periódica.
`debug_keys` (MapScanner)	teste: ligado principal: desligado	Liga a tecla F9 (apagar progresso).
`forca_pulo` / `gravidade` (player)	-350 / 980	Altura do pulo (≈ v²/2g). Muda a relação com os degraus de 80 px.
`max_pulos` (player)	2	1 = só pulo simples; 2 = pulo duplo; 3 = triplo.
`tempo_coyote` (player)	0,1 s	Tolerância para pular logo depois de sair da beirada.
`velocidade_andar / correr`	150 / 250	Velocidade horizontal (correr com Shift).
`distancia / duracao` (plataforma móvel)	340 px / 3 s	Percurso e velocidade da plataforma.
`distancia / velocidade` (inimigo)	varia / 60	Área de patrulha do inimigo.

Sugestões de parâmetros para mostrar no vídeo

1) **cell_size** de 64 → 32 → 128 e mostrar a névoa ficando mais fina/grossa. 2) **vertical_smoothing** 0 vs 8 enquanto pula. 3) **max_pulos** = 1 e mostrar que os degraus de 80 px ficam impossíveis. 4) Trocar a condição da sala secreta no Inspector (ex.: só a porta).

7. Integração e uso: a fase principal

Depois de validado na fase de teste, o MapScanner foi integrado à fase principal (`game.tscn`), que é a cena que abre quando o jogo é executado.



Figura 11 — Fase principal. A fase cabe inteira na câmera, então o minimapa mostra tudo descoberto e centralizado, sem zoom (regra de “mapa pequeno” da especificação).

7.1 Como colocar o MapScanner em qualquer fase (passo a passo)

- Adicionar um nó `Node2D` com o script `map_scanner.gd` e criar uma `MapDefinition` com ID único e os limites da fase.
- Garantir uma `Camera2D` ativa e o jogador no grupo `map_player`.
- Colocar chão/paredes/plataformas no grupo `map_static_terrain`.
- Portas e paredes que mudam: grupo `map_dynamic_terrain` + sinal `map_changed` quando mudarem.
- Plataformas móveis: `map_moving_platforms`. Inimigos: `map_enemies`. Colecionáveis: `map_collectibles`.
- Segredos: adicionar uma `SecretDefinition` na MapDefinition e uma `SecretArea` (retângulo roxo) com o mesmo ID. Alinhar a região à grade.
- Registrar `map_persistence_manager.gd` como Autoload com o nome `MapPersistenceManager` (uma vez por projeto).

Por quê?

Os **grupos** são o “contrato” entre o jogo e o minimapa: o MapScanner não precisa saber o que é uma porta ou um inimigo — só que aquele nó está num grupo. Isso deixa o sistema reutilizável e desacoplado do gameplay (o MapScanner nunca altera portas, paredes ou plataformas; só consulta).

7.2 Correções feitas no jogo durante a integração

Problema encontrado	Correção
Na fase principal, o chão acabava nas bordas da tela e não havia paredes: o jogador caía para sempre.	Paredes invisíveis nas duas pontas + retorno ao ponto inicial se cair abaixo do limite.
Na fase principal não havia câmera nem o jogador no grupo do minimapa.	Camera2D com limites da fase e grupo `map_player`.
Cair da beirada deixava 2 pulos no ar.	Pulo do chão é perdido ao cair (com coyote time de 0,1 s).
A colisão do jogador usava escala não uniforme (1,76 × 1).	Tamanho real 15 × 41 na forma, escala 1 (mesmo tamanho final).
A ação “Atacar” só respondia ao dispositivo 0.	Todos os dispositivos (−1), igual à ação “Pular”.
Na fase de teste, a 1ª plataforma estava a 136 px do chão (impossível de alcançar).	Degraus refeitos com 80 px.
O autoload de save não estava registrado no projeto.	Registrado em `project.godot`.

8. Validação e testes

Além de jogar a cena, foram escritos **testes automáticos** que abrem o jogo sem janela (modo *headless* da Godot 4.6.1), movem o jogador, simulam teclas e conferem o resultado. Eles cobrem a lista de testes da seção 10 da especificação.

O que foi testado	Resultado
Pulo simples sobe 65,5 px (não alcança 80); duplo sobe 130,4 px (alcança)	OK
Subida real: pulo simples não sobe na P1; duplo sobe na P1 e da P1 na P2	OK
Sem eventos, terreno dinâmico não é reconstruído em 60 frames	OK
Porta abre/fecha e o mapa acompanha; parede destruída some; objeto criado em jogo é registrado sem duplicar	OK
Plataforma móvel aparece só na câmera, atualiza a posição, some ao sair e não deixa rastro	OK
Inimigo: aparece só depois de visto; fica translúcido 5 s depois sem expirar; é invalidado ao revisitar; some ao morrer	OK
Colecionável: continua marcado longe; some ao ser coletado; pode ser redescoberto	OK
Segredo: bloqueado mesmo com a câmera cobrindo a sala; AND exige porta + entrada; OR libera com a parede; revela aos poucos (18 de 48 células)	OK
Reinício: esquece entidades, mantém células e segredos; F9 apaga e rebloqueia	OK
Save: versão 2 com segredos, sem entidades nem condições intermediárias; save da versão 1 continua legível	OK
Fase principal: câmera, grupo, chão lido, 198 células, HUD 4:3, cobertura ≥ 2×, centralizado, paredes seguram, queda volta ao início, progresso restaurado	OK
Cair da beirada: sobra 1 pulo no ar; coyote time permite pular logo após sair; ainda sobra o duplo	OK

Total: **51** verificações na fase de teste e **21** na fase principal, todas passando. As capturas deste documento foram geradas com o próprio jogo rodando.

9. Ferramentas, referências e créditos

Item	Uso no projeto
Godot Engine 4.6.1 (GDScript)	Engine do jogo; nós `CanvasLayer`, `Camera2D`, `Control._draw`, `Resource`, grupos, sinais, `FileAccess`, Autoload.
Documentação oficial da Godot — docs.godotengine.org	Referência de API (CanvasItem.draw_*, Camera2D, TileMapLayer, FileAccess/DirAccess, SceneTree.node_added).
Especificação “MapScanner v1.0”	Documento de planejamento com as regras e a arquitetura aprovadas; guiou todas as etapas.
Sprites do Samurai — CraftPix (craftpix.net/freebies)	Animações do jogador. Licença: craftpix.net/file-licenses.
AutoSprite Free Asset Library (CC0 1.0)	Assets livres incluídos no projeto (LICENSE.txt).
Fonte DigitalDisco — dafont.com	Fonte listada nos assets do projeto.
Claude (Anthropic) — assistente de IA	Apoio na implementação dos scripts do MapScanner seguindo a especificação, nos testes automáticos, nas correções de lógica e na redação deste documento. A atividade permite IA desde que creditada.
Python + matplotlib, ReportLab	Geração das figuras e deste PDF.

Conceitos de referência: *fog of war* (névoa de guerra), *coyote time*, gravação atômica de arquivos (escrever em temporário e renomear) e o padrão Observer (sinais da Godot).

10. Dificuldades, decisões e aprendizados

10.1 Dificuldades e como foram resolvidas

Dificuldade	Solução / aprendizado
O progresso não salvava, embora o script de save existisse.	Faltava registrar o Autoload no projeto. **Dica:** um script de Autoload não roda sozinho; ele precisa estar em Projeto → Configurações → Globais.
Plataformas impossíveis de alcançar na cena de teste.	Calcular a altura do pulo ($v^2/2g$) **antes** de posicionar plataformas. O degrau de 80 px ficou entre o pulo simples e o duplo.
A porta abria, mas o minimapa continuava mostrando-a.	Colisão alterada com `set_deferred` só muda no fim do frame. O sinal também foi adiado (`call_deferred`).
Vazamento de memória ao fechar o jogo.	Um `Array[SecretCondition]` dentro da própria classe cria referência cíclica no script. Trocado por `Array[Resource]`.
Testes sem janela davam resultados estranhos.	No modo headless a janela tem 64 × 64 px; a câmera via quase nada. Os testes forçam 1152 × 648.
Primeira versão apagava inimigos após 3 s e relia portas a cada frame.	Ao reler a especificação: inimigos só somem ao revisitar/morrer, e o terreno dinâmico é por eventos. **Aprendizado:** validar cada etapa contra o documento antes de seguir.

10.2 Decisões e interpretações

- **“Descoberta completa do segredo”** foi interpretada como “todas as condições cumpridas” (desbloqueio). É isso que vai para o save.
- **Bloqueio por célula:** qualquer célula que encoste na região secreta fica bloqueada — é conservador (prefere esconder a mais do que revelar). Por isso as salas devem ser alinhadas à grade.

- **Inimigos detectados pelo ponto de origem** (a posição do nó), não pela caixa inteira — simples e suficiente para marcadores.
- **Desenho por retângulos:** colisões redondas ou inclinadas viram o retângulo que as envolve. A especificação pede geometria simplificada.

10.3 Limitações conhecidas

- A rotação da câmera é ignorada no cálculo do retângulo visível.
- Objetos de terreno dinâmico precisam emitir `map_changed`; se não emitirem, mudanças de estado (sem remoção) não aparecem.
- O projeto não usa modo de *stretch*: em telas maiores que 1152 × 648 a fase principal não se adapta. Sugestão: Projeto → Configurações → Exibição → Janela → Stretch = `canvas_items`.
- Cores e símbolos ainda são provisórios (a especificação deixa isso pendente).

10.4 Dicas para quem for implementar algo parecido

- Escreva a regra antes do código (a especificação economizou muito retrabalho).
- Separe “decidir” de “desenhar”: o renderer burro facilita depurar.
- Use grupos como contrato entre sistemas.
- Crie uma cena de teste com um exemplo de cada caso, e atalhos de QA (F6, F7, F9).
- Teste automaticamente o que dá para medir (alturas, contagens, arquivos).

11. Roteiro sugerido para o vídeo-showcase (~5 min)

Tempo	Parte (atividade)	O que mostrar / falar
0:00–0:40	1. Solução	Jogo rodando com o minimapa. “Um minimapa que se desenha sozinho, revela só o que você viu e esconde segredos.” Onde é útil (metroidvanias, exploração).
0:40–1:50	2. Funcionamento	Figura 1 (cadeia). Grade de células (Figura 5): câmera → células descobertas. Terreno estático vs dinâmico. Política: todo frame / 0,1 s / eventos.
1:50–2:50	3. Implementação	Figura 10 (componentes). Mostrar `_ready()` e a ordem de entrada. Grupos como contrato. Persistência segura (.tmp/.bak). O que foi adaptado: degraus de 80 px, coyote time, paredes na fase principal.
2:50–4:20	4. Demonstração	Subir os degraus com pulo duplo; porta F6 sumindo do mapa; plataforma móvel em contorno; inimigo translúcido e invalidado ao voltar; colecionável coletado; sala secreta bloqueada → quebrar parede → revelar aos poucos; fechar e abrir o jogo (progresso mantido). Alterar 2–3 parâmetros no Inspector.
4:20–5:00	5. Aprendizados	Seção 10: Autoload, set_deferred, referência cíclica, headless 64×64, ler a especificação. Limitações. Contribuição de cada integrante.

12. Validação simples do tema (para o acompanhamento)

Item	Resposta
Solução escolhida	Minimapa de exploração (MapScanner) com névoa por grade, terreno dinâmico, entidades temporárias, salas secretas e save.
Problema que resolve	Orientação e exploração em fases 2D grandes, com mundo que muda e áreas secretas, sem desenhar mapas à mão.
Implementação prevista	Sistema funcionando na fase de teste e integrado à fase principal do projeto TDE (Godot 4.6).
Demonstração	Cena de teste com um caso de cada mecânica, atalhos F6/F7/F9 e parâmetros alterados no Inspector.
Equipe	(preencher)
Primeira referência	Especificação MapScanner v1.0 + documentação oficial da Godot.
Principal dúvida	Como revelar salas secretas sem “entregar” a sala inteira de uma vez — resolvido com autorização por célula.

13. Contribuição de cada integrante

Preencha com a divisão real do trabalho (a atividade exige que isso apareça no vídeo).

Integrante	Contribuição
Assistente de IA (Claude)	Apoio na implementação dos scripts, testes automáticos, correções e redação deste documento (ver seção 9).